

Lecture Guide: Comprehensive PHP & MySQLi Database Operations (CRUD)

INSTRUCTIONAL OBJECTIVE:

To provide TYBCA students with a clear, step-by-step practical guide on executing Database Connectivity, Insertion, Update, and Deletion operations using PHP's Object-Oriented MySQLi extension. Optimized for standard laboratory exercises and lecture delivery.

Step 1: Database Setup in MySQL (phpMyAdmin)

Before writing any PHP script, we must establish a backend database environment. Instruct students to launch XAMPP/WAMP, open phpMyAdmin, and execute the following queries to set up a uniform test database table.

```
-- Step 1: Create the Database
CREATE DATABASE bca_db;
USE bca_db;

-- Step 2: Create a simple Student Table
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL
);
```

Step 2: Establishing Database Connection (`db\_connect.php`)

We use the object-oriented MySQLi class approach. Explain to students that separating connection parameters into an isolated file avoids redundancy across multiple project views.

```
<?php
// db_connect.php
$servername = "localhost";
```

```

$username = "root";
$password = ""; // Default XAMPP password is empty
$dbname = "bca_db";

// Create an instance of the mysqli class
$conn = new mysqli($servername, $username, $password, $dbname);

// Verify if the connection succeeded
if ($conn->connect_error) {
    die("Connection failed permanently: " . $conn->connect_error);
}
echo "Database connected successfully!";
?>

```

Step 3: Inserting Data (`insert.php`)

This script outlines how to process hardcoded values directly via standard SQL queries, serving as an ideal introductory demonstration before moving forward to active HTML Form handling.

```

<?php
// insert.php
include 'db_connect.php'; // Include connection block

// Basic student information values
$name = "Rahul Sharma";
$email = "rahul@gmail.com";
$city = "Surat";

// Formulate the dynamic SQL query
$sql = "INSERT INTO students (name, email, city) VALUES ('$name', '$email', '$city')";

// Execute using the query method
if ($conn->query($sql) === TRUE) {
    echo "<br>Record inserted successfully!";
} else {
    echo "<br>Error inserting record: " . $conn->error;
}

// Terminate the active connection link
$conn->close();
?>

```

Step 4: Updating Existing Records (`update.php`)

Explain to students that the UPDATE operation requires an explicit WHERE condition statement, or else the entire database sequence runs the risk of getting altered unintentionally.

```
<?php
// update.php
include 'db_connect.php';

// New values to update target index
$target_id = 1;
$new_city = "Mumbai";

// Structured string query
$sql = "UPDATE students SET city='$new_city' WHERE id=$target_id";

if ($conn->query($sql) === TRUE) {
    echo "<br>Record updated successfully!";
} else {
    echo "<br>Error updating record: " . $conn->error;
}

$conn->close();
?>
```

Step 5: Deleting Records (`delete.php`)

Demonstrate how rows are permanently dropped using a unique identifier primary key attribute. Instruct students to always execute updates and deletes cautiously.

```
<?php
// delete.php
include 'db_connect.php';

// Specific record targeted for deletion
$target_id = 1;

$sql = "DELETE FROM students WHERE id=$target_id";

if ($conn->query($sql) === TRUE) {
    echo "<br>Record deleted successfully!";
} else {
    echo "<br>Error deleting record: " . $conn->error;
}
```

```
}
```

```
$conn->close();
```

```
?>
```